

Table of Contents

Отчёт по лабораторной работе 3

Тема: Разработка одностраничного веб-приложения (SPA) с использованием Vue.js

Проект: DataFlow — панель управления data pipelines

Студент: Карташов Игорь, группа J3212

Ветка: lab3-kartashov (наследуется от lab2-kartashov)

Расположение: J3212/Карташов Игорь/homeworks/lab-3/dataflow-vue/

1. Цель работы

Мигрировать ранее разработанное приложение DataFlow (ЛР1 и ЛР2) с многостраничной реализации на HTML + vanilla JavaScript в одностраничное приложение (SPA) на фреймворке **Vue 3**, сохранив функциональность и улучшив архитектуру за счёт компонентного подхода и composables.

2. Исходное состояние (ЛР1 / ЛР2)

В ЛР1/ЛР2 приложение состояло из отдельных HTML-страниц:

Страница	Назначение
login.html / register.html	Авторизация и регистрация
index.html	Список pipelines с поиском и фильтрами
pipeline-detail.html	Детали pipeline: DAG-граф, логи, история
profile.html	Профиль пользователя, статистика, connections

Данные загружались через **fetch** с **json-server** (<http://localhost:3000>), логика была разбита по файлам api.js, auth.js, index.js, pipeline-detail.js, profile.js, theme.js.

Навигация — полная перезагрузка страницы через `<a href="...">`.

3. Что сделано в ЛР3

3.1. Создан Vue-проект

Инициализирован проект через **Vite** с шаблоном Vue 3:

```
lab-3/dataflow-vue/
├── db.json          # mock API (скопирован из lab-1)
├── src/
│   ├── api/        # слой работы с API
│   ├── composables/ # переиспользуемая логика
│   ├── components/  # UI-компоненты
│   ├── views/       # страницы SPA
│   ├── router/      # маршрутизация
│   ├── assets/      # стили (style.css, red.css)
│   ├── App.vue
│   └── main.js
├── index.html
└── package.json
```

Стек технологий: - Vue 3 (Composition API, <script setup>) - Vue Router 4 - Axios - Bootstrap 5 + Bootstrap Icons - Vite (сборка и dev-сервер) - json-server (mock REST API)

4. Маршрутизация (Vue Router)

Подключён **vue-router** с history-режимом. Вместо отдельных HTML-файлов — маршруты SPA:

Маршрут	Компонент	Доступ
/login	LoginView	Только для гостей
/register	RegisterView	Только для гостей
/	PipelinesView	Требует авторизации
/pipelines/:id	PipelineDetailView	Требует авторизации
/profile	ProfileView	Требует авторизации

Navigation guard (router.beforeEach): - неавторизованный пользователь перенаправляется на /login; - авторизованный пользователь не может зайти на /login и /register — редирект на главную.

Сессия хранится в localStorage (userId), как в оригинальной ЛР1.

5. Работа с внешним API (Axios)

Вместо fetch используется **axios**.

src/api/client.js — базовый экземпляр:

```
axios.create({ baseURL: 'http://localhost:3000' })
```

src/api/index.js — методы по сущностям: - pipelinesApi — список и детали pipelines - tasksApi — задачи pipeline - logsApi — логи - runsApi — история запусков - usersApi — пользователи (login, register) - connectionsApi — подключения пользователя

Mock-сервер запускается командой `npm run api` (json-server на порту 3000, данные из db.json).

6. Composables — выделение повторяющейся логики

Composable	Назначение
useAuth	Авторизация: login, register, logout, проверка сессии, хранение userId
useTheme	Переключение тем light → dark → red, сохранение в localStorage
useStatus	Общая логика статусов: CSS-классы badge и текстовые метки (success/failed/running)

Composables используются в нескольких компонентах и views, что убирает дублирование кода из оригинальных auth.js, theme.js и повторяющихся STATUS_CLASSES в JS-файлах LP1.

7. Компонентная архитектура

Layout (общая оболочка)

- **AppLayout** — sidebar + topbar + <slot> для контента
- **AppSidebar** — боковое меню с RouterLink
- **AppTopbar** — заголовок, breadcrumb, ссылка на профиль / выход
- **ThemeToggle** — кнопка смены темы

Страницы (views)

- **LoginView / RegisterView** — формы авторизации

- **PipelinesView** — главная: фильтры + таблица
- **PipelineDetailView** — детали pipeline с вкладками
- **ProfileView** — профиль, статистика, connections

Переиспользуемые UI-компоненты

- **PipelineFilterBar** — поиск и фильтры по статусу
- **PipelineTable** — таблица pipelines
- **StatusBadge** — badge статуса
- **DagGraph** — визуализация DAG (граф задач)
- **LogTerminal** — вывод логов в терминальном стиле
- **RunHistoryTable** — таблица истории запусков
- **TaskModal** — модальное окно с информацией о задаче
- **ProfileCard** — карточка профиля
- **StatsPanel** — блок статистики (DAGs, Runs, Success Rate)
- **ConnectionList** — список подключений

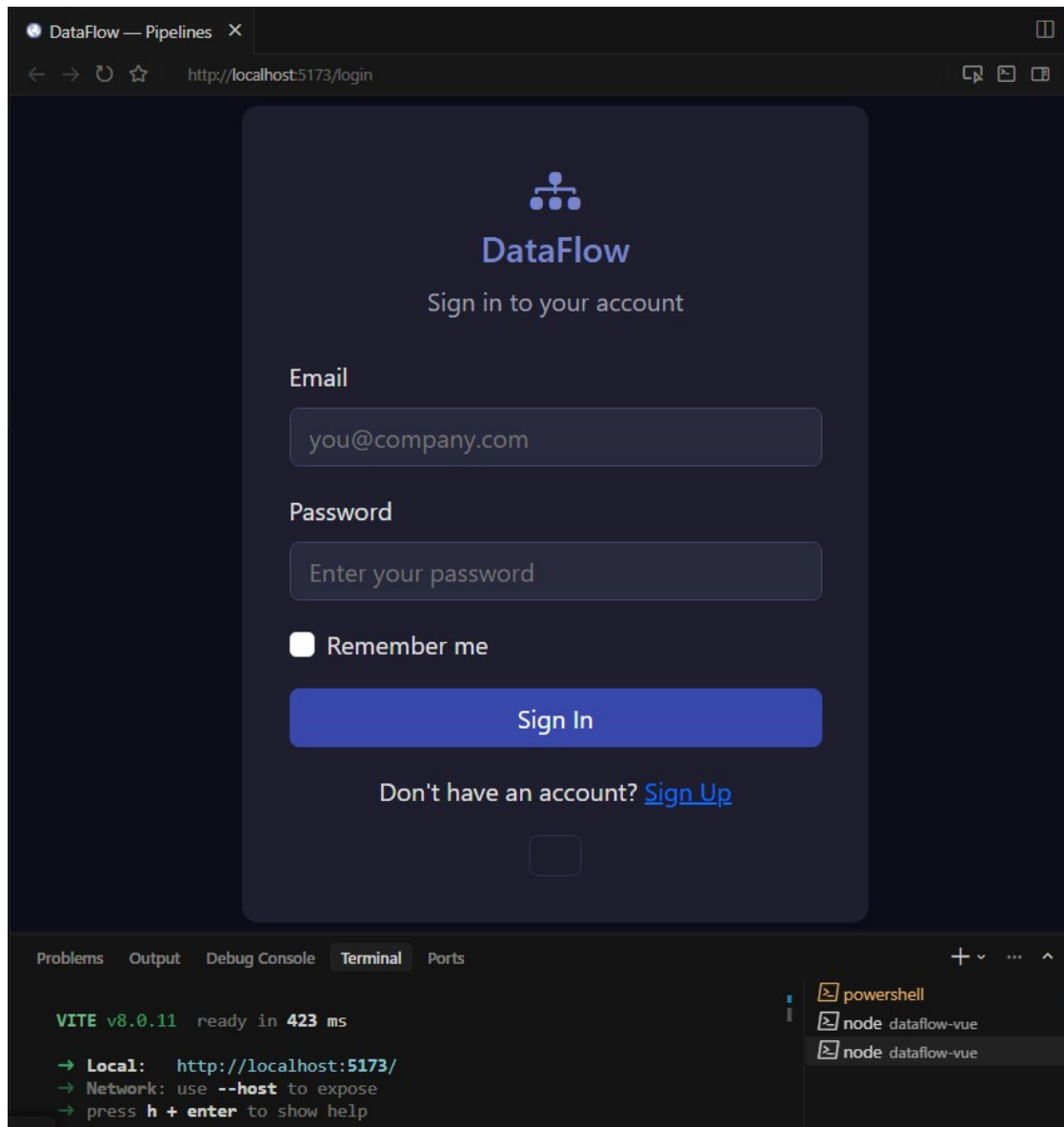
Каждый компонент отвечает за одну задачу: layout отделён от бизнес-логики, таблицы — от фильтров, граф — от логов.

8. Перенесённая функциональность

Функция из ЛР1/ЛР2	Реализация в Vue SPA
Список pipelines	PipelinesView + PipelineTable + PipelineFilterBar
Поиск и фильтрация	Реактивные ref + computed вместо DOM-манипуляций
Детали pipeline	PipelineDetailView с вкладками Graph / Logs / History
DAG-граф	DagGraph — группировка задач по order, параллельные ветки
Модалка задачи	TaskModal по клику на блок DAG
Login / Register	useAuth + формы во views
Профиль и connections	ProfileView + ProfileCard + ConnectionList
Темы (light/dark/red)	useTheme + CSS-переменные из style.css / red.css
Защита маршрутов	requireAuth() заменён на router guard

9. Запуск и проверка

Ниже приведён пример экрана авторизации приложения DataFlow в режиме разработки.



Экран авторизации DataFlow

```
cd "J3212/Карташов Игорь/homeworks/lab-3/dataflow-vue"
```

```
npm install
```

```
npm run api # json-server → http://localhost:3000
```

```
npm run dev # Vite → http://localhost:5173
```

Тестовый аккаунт: igor@company.com / admin123

Сборка production: npm run build — проходит без ошибок.

10. Сравнение: было → стало

Аспект	ЛР1/ЛР2 (vanilla JS)	ЛР3 (Vue SPA)
Навигация	Перезагрузка HTML-страниц	Клиентский роутинг без перезагрузки
API	fetch + функции api()	Axios + модульный API-слой
UI	Ручной DOM (innerHTML, createElement)	Декларативные шаблоны Vue
Состояние	Глобальные переменные, DOM	Реактивные ref / computed
Повторяющийся код	Копирование в каждый JS-файл	Composables и компоненты
Авторизация	requireAuth() на каждой странице	Единый router guard

11. Выводы

В рамках лабораторной работы 3 приложение **DataFlow** успешно мигрировано на Vue 3. Выполнены все требования:

1. **Vue Router** — настроена маршрутизация с защитой маршрутов.
2. **Axios** — централизованный API-слой для работы с json-server.
3. **Компонентный подход** — UI разбит на layout, views и переиспользуемые компоненты.
4. **Composables** — авторизация, темы и статусы вынесены в отдельные файлы.

SPA сохраняет весь функционал предыдущих лабораторных работ и получает более поддерживаемую архитектуру: изменения в одном composible или компоненте автоматически применяются везде, где он используется.

Если нужно — могу оформить это в PDF или сохранить как .md-файл в папку lab-3.